Grundlagen

Alphabete

- $\Sigma = \{a, b, c\}$: Mengen von drei Symbolen
- $\Sigma_{\text{Bool}} = \{0, 1\}$: Boolesches Alphabet
- $\mathbb{N}, \mathbb{R}, \mathbb{Z}$ sind **keine** Alphabete (∞ Mächtigkeit)

Wörter

- abc: Wort über Σ_{lat} (oder $\Sigma = \{a, b, c\}$)
- 100111: Wort über Alphabet $0, 1$
- ε: Leeres Wort (über jede Alphabet)

Sprachen

Eine Teilmenge $L \subseteq \Sigma^*$ von Wörtern über einem Alphabet Σ wird als **Sprache über** Σ bezeichnet.

- Wenn $\Sigma_1 \subseteq \Sigma_2$ gilt und L eine Sprache über Σ_1 ist, dann ist L auch eine Sprache über Σ_2.
- Σ^* ist eine Sprache über jedem Alphabet Σ.
- $\{\} = \emptyset$ ist die *leere Sprache* für jedes Alphabet.
- $\{\varepsilon\}$ ist die Sprache, die aus dem leeren Wort ε besteht für jedes Alphabet Σ (Anmerkung: $\emptyset \neq \{\varepsilon\}$).

Menge von Wörter

- $\Sigma^* = \underbrace{\Sigma^0}_{1} \cup \underbrace{\Sigma^1}_{2} \cup \underbrace{\Sigma^2}_{4} \cup \dots$ (Kleensche Hülle)
- $\Sigma^+ = \underbrace{\Sigma^1}_{2} \cup \underbrace{\Sigma^2}_{4} \cup \underbrace{\Sigma^3}_{8} \dots = \Sigma^* \setminus \{\varepsilon\}$ (Pos. Hülle)

Konkatenation: Verkettung von zwei beliebigen Wörtern x und y

$x \circ y = xy := (x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_m)$

Wortpotenzen:

- $x^{n+1} = x^n \circ x = x^n x$
- $bbababababbaaaabab = b^2(ab)^4ba^3(ab)^2$

Reguläre Ausdrücke

Es sei Σ ein beliebiges Alphabet. Die Sprache RA_Σ der **regulären Ausdrücke** über Σ ist wie folgt definiert:

- $\varnothing, \epsilon \in \text{RA}_\Sigma$
- $\Sigma \subset \text{RA}_\Sigma$
- $R \in \text{RA}_\Sigma \Rightarrow (R^*) \in \text{RA}_\Sigma$
- $R, S \in \text{RA}_\Sigma \Rightarrow (RS) \in \text{RA}_\Sigma$
- $R, S \in \text{RA}_\Sigma \Rightarrow (R|S) \in \text{RA}_\Sigma$

Für jeden regulären Ausdruck $R \in RA_\Sigma$ definieren wir die Sprache $L(R)$ wie folgt:

- $L(\emptyset) = \emptyset \Rightarrow$ Leere Sprache
- $L(\varepsilon) = \varepsilon \Rightarrow$ Sprache, die nur das leere Wort enthält
- $L(a) = \{a\}$ für $a \in \Sigma \Rightarrow$ Beschreibt die Sprache $\{a\}$
- $L(R^*) = L(R)^* \Rightarrow$ Kombinierte Wörter von R
- $L(R|S) = L(R) \cup L(S) \Rightarrow$ Wörter die von R oder S beschrieben werden
- $L(RS) = L(R)L(S) \Rightarrow$ Verkettungen von Wörtern (R = Präfix)

Eine Sprache A über dem Alphabet Σ heisst **regulär**, falls $A = L(R)$ für einen regulären Ausdruck $R \in RA_\Sigma$ gilt.

Beispiele:

- $R_1 = a^*b \Rightarrow L(R_1) = \{b, ab, aab, \dots\}$
- $R_2 = (aa)^*b^*aba \Rightarrow L(R_2) = \{aba, baba, aaaba, aababa, \dots\}$
- $R_3 = (a|ab)^* \Rightarrow L(R_3) = \{\varepsilon, a, ab, aa, abab, \dots\}$

Die Menge RA_Σ ist eine Sprache über dem Alphabet $\{\emptyset, \varepsilon, *, (,), |\} \cup \Sigma$.

Priorisierung von Operatoren:

1. * = Wiederholung
2. Konkatenation
3. $|$ = Auswahl

Beispiele:

- $(aa)^*b^*aba = (aa)^*b^*aba$
- $(ab)|(ba) = ab|ba$
- $a(b(ba))|b = abba|b$

Erweiterte Syntax:

- $R^+ = R(R^*)$
- $R? = (R|\varepsilon)$

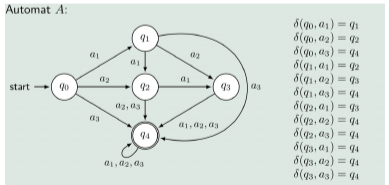
Endliche Automaten

DEA

Ein **deterministischer endlicher Automat** ist ein 5-Tupel $M = (Q, \Sigma, \delta, q_0, F)$ mit

- Menge von **Zustände** $Q = \{q_0, \dots, q_n\}$
- **Eingabealphabet** $\Sigma = \{a_1, \dots, a_m\}$
- **Übergangsfunktion** δ: $Q \times \Sigma \to Q$
- **Startzustand** $q_0 \in Q$
- Menge **akzept. Zustände** $F \subseteq Q$

Automat A:



$\delta(q, a) = p$ bedeutet: EA wechselt zu p, falls in q Symbol a gelesen wird.

Sei M ein EA. Eine **Konfiguration** von M auf ω ist ein Element aus $Q \times \Sigma^*$.

- Startkonfiguration von M auf $\omega \Rightarrow \{q_0, \omega\} \in \{q_0\} \times \Sigma^*$
- Endkonfiguration $\Rightarrow (q_n, \varepsilon)$

c

M

Nichtdeterminismus

Bei ***deterministischen*** **endlichen Automaten** folgt jede Konfiguration **eindeutig** aus der vorhergehenden Konfiguration.

Bei ***nichtdeterministischen*** endlichen Automaten können auf eine Konfiguration entweder **keine**, **genau eine** oder **mehrere** unterschiedliche Konfigurationen folgen.

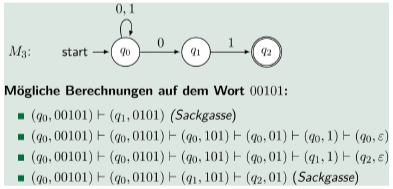
NEA

Ein nichtdeterministischer endlicher Automat (**NEA**) ist ein Quintupel
$M = (Q, \Sigma, \delta, q_0, F)$

Der einzige Unterschied zum DEA besteht in der Übergangsfunktion δ.

- Übergangsfunktion δ: $Q \times \Sigma \to \mathcal{P}(Q)$

Ein ε-NEA erlaubt zusätzlich noch ε Übergänge.



DEAs, NEAs, ε-NEAs und RAs sind **gleichmächtig**.

Reguläre Sprachen

Reguläre Sprachen sind durch verschiedene äquivalente Mechanismen darstellbar

- Akzeptierender Mechanismus: DEA, NEA, ε-NEA
- Beschreibender Mechanismus: RA

Seien L_1 und L_2 zwei reguläre Sprachen über Σ. Dann ist die Vereinigung ebenfalls regulär. $L_1 \cup L_2 = \{\omega \mid \omega \in L_1 \vee \omega \in L_2\}$

Seien L_1 und L_2 zwei reguläre Sprachen über Σ. Dann sind die **Konkatenation** $L_1 \cdot L_2 = \{\omega = \omega_1\omega_2 \mid \omega_1 \in L_1 \wedge \omega_1 \in L_2\}$ und die **Kleenesche Hülle** $L^* = \{\omega = \omega_1 \dots \omega_n \mid \omega_i \in L$ für alle $i \in \{1, \dots, n\}\}$ auch regulär.

Sei L eine reguläre Sprache über Σ. Dann ist auch das **Komplement** $\overline{L} = \Sigma^* - L = \{w \in \Sigma^* \mid w \notin L\}$ regulär.

Kontextfreie Grammatiken

Eine **kontextfreie Grammatik** G (KFG) ist ein 4-Tupel (N, Σ, P, A) mit

- N ist das Alphabet der **Nichterminale** (Variablen).
- Σ ist das Alphabet der **Terminale**.
- P ist eine endliche Menge von **Produktionen** (Regeln). Jede Produktion hat die Form $X \to \beta$ mit **Kopf** $X \in N$ und Rumpf $\beta \in (N \cup \Sigma)^*$.
- A ist das **Startsymbol**, wobei $A \in N$.

Seien α, β und γ Satzformen und $A \to \gamma$ eine Produktion.

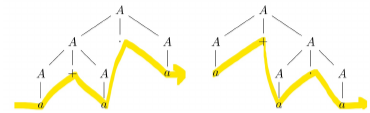
- **b**

$\dots \Rightarrow w.$

Eine Ableitung kann als **Ableitungsbaum**/Parsebaum dargestellt werden. Z.B. Ableitungsbaum für das Wort (())().

$G_3 = \{\{A\}, \{a, \cdot, +, (,)\}, P, A\}$, mit $P = \{A \to A + A \mid A \cdot A \mid (A) \mid a\}$
(a steht vereinfachend für eine beliebige Binärzahl)

Das Wort $a + a \cdot a$ kann durch zwei verschiedene Ableitungsbäume dargestellt werden:



- Korrekte Klammerung erzwingen
- Grammatik anpassen
- Den Produktionen Vorrang vergeben

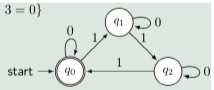
Jede reguläre Sprache kann durch eine KFG beschrieben werden.

Sei L eine reguläre Sprache. Dann gibt es einen DEA $M = (Q, \Sigma, \delta, q_0, F)$ mit $L(M) = L$. Wir können einen KFG für L wie folgt bauen:

- Für jeden Zustand q_i gibt es ein Nichtterminal Q_i
- Für jede Transition $\delta(q_i, a) = q_j$ erstellen wir die Produktion $Q_i \to aQ_j$
- Für jeden akzeptierenden Zustand $q_i \in F$ erstellen wir die Produktion $Q_i \to \varepsilon$
- Das Nichtterminal Q_0 wird zum Startsymbol A

$L_5 = \{w \in \{0,1\}^* \mid |w|_1 \mod 3 = 0\}$

Nichtterminale:
Q_0, Q_1, Q_2



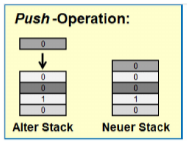
Produktionen:
$Q_0 \to 0Q_0 \mid 1Q_1 \mid \varepsilon$
$Q_1 \to 0Q_1 \mid 1Q_2$
$Q_2 \to 0Q_2 \mid 1Q_0$

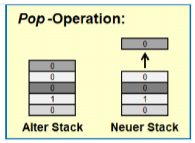
Beispiel für Ableitung von $w = 10011$:
$Q_0 \Rightarrow 1Q_1 \Rightarrow 10Q_1 \Rightarrow 100Q_1 \Rightarrow 1001Q_2 \Rightarrow 10011Q_0 \Rightarrow 10011$

Kellerautomaten

Ein **Kellerautomat** ist ein EA für die Erkennung von kontextfreien Sprachen mit unbegrenztem Stack (Keller, Stapel).

Kelleroperationen:



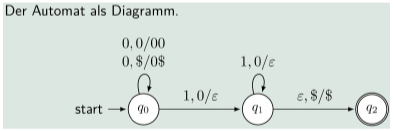


Ein **deterministischer Kellerautomat** M ist ein 7-Tupel $(Q, \Sigma, \Gamma, \delta, q_0, \$, F)$ mit

- Q ist die endliche Menge von Zustände
- Σ ist das Alphabet der Eingabe
- Γ ist das Alphabet des Kellers
- $\delta : Q \times (\Sigma \cup \varepsilon) \times \Gamma \to Q \times \Gamma^*$ ist eine (partielle) Übergangsfunktion
- $q_0 \in Q$ ist der Startzustand
- $\$ \in \Gamma$ ist ein ausgezeichnetes Symbol vom Alphabet des Kellers
- $F \subseteq Q$ ist die Menge der akzeptierenden Zustände

Anfangs enthält der Keller eine Instanz des Symbols $\$$.

Der Automat als Diagramm.

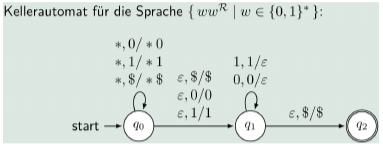


Ein **nichtdeterministischer Kellerautomat** unterscheidet sich nur in der Übergangsfunktion:
$\delta : Q \times (\Sigma \cup \varepsilon) \times \Gamma \to \mathcal{P}(Q \times \Gamma^*)$

- er
..
- ..

Beispiel NKA:

Kellerautomat für die Sprache $\{ ww^{\mathcal{R}} \mid w \in \{0,1\}^* \}$:



- ra

-

Turingmaschinen (TM)

Eine TM ist informell ein endlicher Automat, ergänzt mit einem Lese-/Schreibkopf und ein unendliches Band von Zellen mit Leerzeichen. Die Eingabe steht am Anfang auf dem Band.

Eine **deterministische TM** ist ein 7-Tupel $M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$ mit

- *endlichen* Menge von Zuständen $Q = \{q_0, \dots, q_n\}$
- Eingabealphabet $\Sigma = \{a_1, \dots, a_m\}$
- Übergangsfunktion $\delta : Q \times \Gamma \to Q \times \Gamma \times D, D = \{L, R\}$
- Startzustand $q_0 \in Q$
- Menge von akzeptierenden Zuständen $F \subseteq Q$
- Bandalphabet Γ (endliche Menge von Symbolen) und $\Sigma \subset \Gamma$
- Leerzeichen $\sqcup$ mit $\sqcup \in \Gamma$ und $\sqcup \notin \Sigma$

Die Übergangsfunktion δ bildet das 2-Tupel (q, X) auf das Tripel (p, Y, D) ab

- $q, p \in Q$ und $X, Y \in \Gamma$
- D ist die Bewegung des Lese-/Schreibkopfes über dem Band und nimmt die Werte L (Links) oder R (Rechts) an.

Vergleich Übergangsfunktion

Endliche Automat (EA):

$\delta(q_1, a) = (q_2)$: $q_1 \xrightarrow{a} q_2$

Kellerautomat (KA):

$\delta(q_1, a, b) = (q_2, w)$: $q_1 \xrightarrow{a, b/w} q_2$

Turing-Maschine (TM):

$\delta(q_1, X) = (q_2, Y, D)$: $q_1 \xrightarrow{X/Y, D} q_2$

Berechnungsmodelle

Es sei $T = (Q, \Sigma, \Gamma, \dots)$ eine Turing-Maschine. Wir können T wie folgt als (partielle) Funktion auffassen:

$$T : \Sigma^* \to \Gamma^*$$

wobei

$$T(w) = \begin{cases} u & \text{falls } T \text{ auf } w \in \Sigma^* \text{ angesetzt, nach endlich vielen Schritten mit } u \text{ auf dem Band anhält,} \\ \uparrow & \text{falls } T \text{ bei Input } w \in \Sigma^* \text{ nicht anhält.} \end{cases}$$

Grundfunktionen

Primitiv rekursiven Grundfunktionen:

- $\forall n \in \mathbb{N}$ und jede Konstante $k \in \mathbb{N}$ die n-stellige *konstante Funktion* $c_k^n = \mathbb{N}^n \to \mathbb{N}$ mit $c_k^n(x_1, \dots, x_k) = k$
- Die *Nachfolgerfunktion* $\eta : \mathbb{N} \to \mathbb{N}$ mit $\eta(x) = x + 1$
- $\forall n \in \mathbb{N}$ und jedes $1 < k < n$ die n-stellige *Projektion* auf die k-te Komponente $\pi_k^n = \mathbb{N}^n \to \mathbb{N}$ mit $\pi_k^n(x_1, \dots, x_k, \dots, x_n) = k$

Loop (Primitiv-Rekursiv)

- Zuweisungen: $x = y + c$ und $x = y - c$
- Sequenzen: P und $Q \to P; Q$
- Schleifen: $P \to$ `Loop` x `Do` P `End`

Beispiele:

Die Addition von natürlichen Zahlen ist LOOP-berechenbar. Das **LOOP-Programm**

```
Loop x1 Do
    x2 = x2 + 1
End;
x0 = x2 + 0
```

berechnet die Addition $Add(x, y) = x + y$.

Die Multiplikation von natürlichen Zahlen ist LOOP-berechenbar. Das **LOOP-Programm**

```
Loop x2 Do
    Loop x1 Do
        x0 = x0 + 1
    End
End
```

berechnet die Multiplikation $Mul(x, y) = x \cdot y$.

While (Turing-complete)

- Jedes `Loop`-Programm ist auch ein `While`-Programm
- `While`-Programme terminieren nicht immer (Endlos-Schleife)

GOTO (Turing-complete)

- Zuweisungen:
  `Mk` : $x_i = x_j + c$
  `Mk` : $x_i = x_j - c$
- Sprunganweisung:
  `Mk: If` $x_i = c$ `Then Goto Mr`
  `Mk: Goto Mr`
- Halt-Anweisung:
  `Mk: HALT`

Das GOTO-Programm

```
M1: x0 = x1 + 0;
M2: If x2 = 0 Then Goto M6;
M3: x2 = x2 - 1;
M4: x0 = x0 + 1;
M5: Goto M2;
M6: Halt
```

berechnet die Addition $Add(x, y) = x + y$.

Entscheidbarkeit

Entscheidbarkeit

Eine Sprache $A \subset \Sigma^*$ heisst **entscheidbar**, wenn eine Turingmaschine T existiert, die das Entscheidungsproblem (Σ, A) löst.

- Ist eine Sprache $A \subset \Sigma^*$ entscheidbar, dann gibt es eine Turingmaschine T, die sich wie folgt verhält:
  - Wenn T mit Bandinhalt $x \in A$ startet, dann hält T nach endlich vielen Schritten mit Bandinhalt "1" (Ja) an.
  - Wenn T mit Bandinhalt $x \in \Sigma^* \backslash A$ startet, dann hält T nach endlich vielen Schritten mit Bandinhalt "0" (Nein) an.
- Insbesondere muss die Turingmaschine T bei jeder Eingabe $x \in \Sigma^*$ nach endlich vielen Schritten halten.

Semi-Entscheidbarkeit

Eine Sprache $A \subset \Sigma^*$ heisst **semi-entscheidbar**, wenn eine Turingmaschine T existiert, die sich wie folgt verhält:

- Wenn T mit Bandinhalt $x \in A$ gestartet wird, dann hält T nach endlich vielen Schritten mit Bandinhalt "1" (Ja) an.
- Wenn T mit Bandinhalt $x \in \Sigma^* \backslash A$ gestartet wird, dann hält T nie an.

Jede entscheidbare Sprache ist auch semi-entscheidbar.

Satz:

- Ist $A \subset \Sigma^*$ eine entscheidbare Sprache, dann ist auch $\overline{A}$ entscheidbar.
- Sind A, B (semi-) entscheidbare Sprachen, dann sind auch $A \cup B$ und $A \cap B$ (semi-) entscheidbar.

Eine Sprache $A \subset \Sigma^*$ ist genau dann entscheidbar, wenn sowohl A als auch $\overline{A}$ semi-entscheidbar ist.

Reduktion

Wir können jede Instanz des Problems P_1 zu einer Instanz des Problems P_2 umformulieren. Das nennt man eine **Reduktion** von P_1 auf P_2.

Eine Sprache $A \subset \Sigma^*$ heisst auf eine Sprache $B \subset \Gamma^*$ **reduzierbar**, wenn es eine totale, Turing-berechenbare Funktion $F : \Sigma^* \to \Gamma^*$ gibt, sodass für alle $w \in \Sigma^*$ gilt:
$w \in A \Leftrightarrow F(w) \in B$
Ist die Sprache A auf die Sprache B reduzierbar, dann schreiben wir $A \preceq B$.

Satz (Transitivität): Für beliebige Sprachen A, B und C gilt:
$A \preceq B \cap B \preceq C \Rightarrow A \preceq C$

Satz: Für beliebige Sprachen $A \subset \Sigma^*$, $B \subset \Gamma^*$ gilt: Ist B (semi-) entscheidbar und $A \preceq B$, dann ist auch A (semi-) entscheidbar.

Halteproblem

Allgemeines Halteproblem H

Gegeben: Der Code $w \in \{0,1\}^*$ einer Turingmaschine T_w und ein Input x.

Gefragt: Hält die Turingmaschine T_w an, wenn man sie auf x ansetzt?

Das **allgemeine Halteproblem** ist die Sprache $H := \{w\#x \in \{0,1,\#\}^* \mid T_w$ angesetzt auf x hält$\}$.

Die Funktion des Zeichens # ist das Trennen des Input-Strings in zwei Inputs.

Leeres Halteproblem H_0

Gegeben: Der Code $w \in \{0,1\}^*$ einer Turingmaschine T_w.

Gefragt: Hält die Turingmaschine T_w an, wenn man sie auf das leere Band ansetzt?

Das **allgemeine Halteproblem** ist die Sprache $H_0 := \{w \in \{0,1\}^* \mid T_w$ angesetzt auf das leere Band hält$\}$.

Spezielles Halteproblem H_S

Gegeben: Der Code $w \in \{0,1\}^*$ einer Turingmaschine T_w.

Gefragt: Hält die Turingmaschine T_w an, wenn man sie auf ihren eigenen Code w (als Input) ansetzt?

Das **allgemeine Halteproblem** ist die Sprache $H_S := \{w \in \{0,1\}^* \mid T_w$ angesetzt auf w hält$\}$.

H, H_0 und H_S sind semi-entscheidbar.

Komplexitätstheorie

Theorie der quantitativen Gesetze und Grenzen der algorithmischen Informationsverarbeitung.

- Zeitkomplexität: Laufzeit des besten Programms, welches das Problem löst
- Platzkomplexität: Speicherplatzbedarf des besten Programms
- Beschreibungskomplexität: Länge des kürzesten Programms

Sei M eine TM, die immer hält, und sei $\omega \in \Sigma^*$. Der Zeitbedarf von M auf der Eingabe ω ist:
$\texttt{Time}_M(\omega)$ = Anzahl von Konfigurationsübergängen der Berechnung von M auf ω.

Der **Zeitbedarf** von M auf Eingaben der Länge $n \in \mathbb{N}$ im schlechtesten Fall ist definiert als:
$\texttt{Time}_M(n) = \max\{\texttt{Time}_M(\omega) \mid |\omega| = n\}$

Asymptotische Komplexität

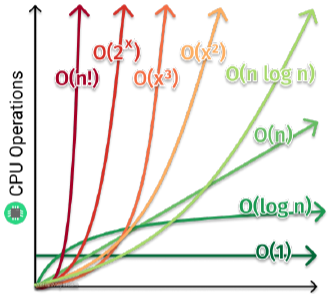
Seien $f, g : \mathbb{N} \to \mathbb{N}$ zwei Funktionen. Dann gilt:

- $f \in \mathcal{O}(g)$: Es existieren $n_0 \in \mathbb{N}$ und $c \in \mathbb{N}$, sodass $\forall n \geq n_0$ gilt:
  $f(n) \leq c \cdot g(n)$
  (f wächst asymptotisch nicht schneller als g)
- $f \in \Omega(g)$: Es existieren $n_0 \in \mathbb{N}$ und $d \in \mathbb{N}$, sodass $\forall n \geq n_0$ gilt:
  $f(n) \geq \frac{1}{d} \cdot g(n)$ (f wächst asymptotisch mindestens so schnell wie g)
- $f \in \Theta(g)$: Es gilt $f(n) \in \mathcal{O}(g(n))$ und $f(n) \in \Gamma(g(n))$
  (f und g sind asymptotisch gleich)

Rechenregeln:

- Konstante Vorfaktoren kann man ignorieren: $c \cdot f(n) \in \mathcal{O}(f(n))$
- Für eine Konstante c gilt: $c \in \mathcal{O}(1)$
- Bei Polynomen ist nur die höchste Potenz entscheidend:
  $a_k n^k + a_{k-1} n^{k-1} + \cdots + a_1 n + a_0 \in \mathcal{O}(n^k)$
- Die $\mathcal{O}$-Notation ist transitiv:
  Aus $f(n) \in \mathcal{O}(g(n))$ und $g(n) \in \mathcal{O}(h(n))$ folgt $f(n) \in \mathcal{O}(h(n))$

Bemerkung: $\mathcal{O}(g(n))$ ist die *Menge* aller Funktionen $f(n)$, für die es zwei Konstanten n_0 und c gibt, sodass $\forall n \geq n_0$ gilt: $f(n) \leq c \cdot g(n)$



Klassifizierung von Problemen

Ein Problem U heisst **in Polynomzeit lösbar**, wenn es eine obere Schranke $\mathcal{O}(n^c)$

Satz: NP ist die Menge aller Sprachen, für die ein Polynomzeit-Verifizierer existiert.

P = Lösung *finden* in Polynomzeit
NP = Lösung *verifizieren* in Polynomzeit

NP-Schwer & NP-vollständig

Eine Sprache L heisst **NP-Schwer**, falls es für alle Sprachen $L' \in NP$ gilt: $L' \preceq_p L$

D.h. L ist bezüglich der Lösbarkeit in polynomieller Zeit mindestens so schwer wie jedes einzelne Problem in NP

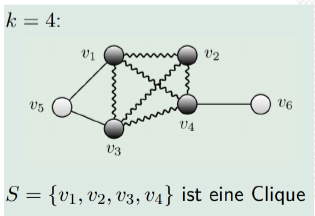
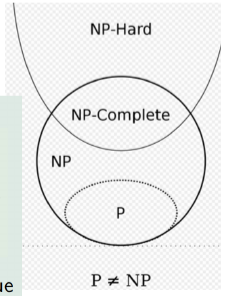
Eine Sprache L heisst **NP-vollständig**, falls $L \in NP$ und L NP-schwer ist.

Die Menge der NP-vollständigen Sprachen betrachten wir als die gesuchte Teilklasse von schwersten Entscheidungsproblemen in NP.

Nachweis der NP-Schwere

1. Finde ein erstes NP-schweres Problem
2. Finde eine Polynomzeitreduktion von einem bekannten NP-schweren Problem auf das neue Problem, dessen NP-Schwere gezeigt werden soll.

Nachweis der NP-Vollständigkeit

1. Zeige, dass das Problem NP-Schwer ist.
2. Zeige, dass das Problem in NP liegt.

Satz: Wenn P_1 NP-schwer und P_2 in NP enthalten ist und eine polynomielle Reduktion $P_1 \preceq_p P_2$ existiert, dann ist P_2 NP-vollständig.